

Nivel 1

Ejercicio 1

Dados los archivos sql (estructura_dades y dades_introduir), ejecuto los documentos, y se crea la base de datos “transacciones”, con dos tablas (“transaction” y “company”).

Una vez ejecutados e importados los archivos, selecciono la base de datos con USE transactions y uso el operador COUNT() para ver el número de filas que contiene cada uno.

```
1 • SELECT * FROM transactions.company;
2 • USE transactions;
3 • SELECT COUNT(*) AS total_company
4 • FROM company;
5 • SELECT COUNT(*) AS total_transaction
6 • FROM transaction;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	total_company
▶	101

Result 11 x

Output

Action Output

#	Time	Action	Message
✓ 1	14:15:57	SELECT * FROM transactions.company LIMIT 0, 1000	101 row(s) returned
✓ 2	14:16:00	SELECT COUNT(*) AS total_company FROM company LIMIT 0, 1000	1 row(s) returned

```
1 • SELECT * FROM transactions.company;
2 • USE transactions;
3 • SELECT COUNT(*) AS total_company
4 • FROM company;
5 • SELECT COUNT(*) AS total_transaction
6 • FROM transaction;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	total_transaction
▶	100001

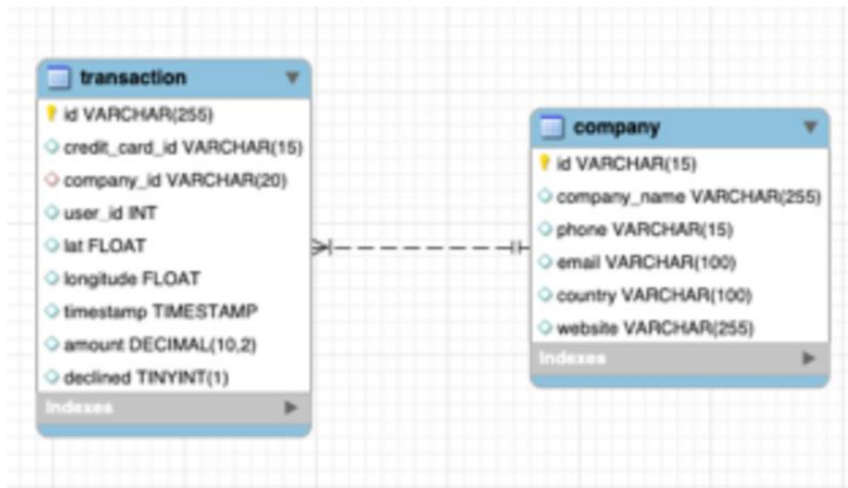
Result 12 x

Output

Action Output

#	Time	Action	Message
✓ 1	14:15:57	SELECT * FROM transactions.company LIMIT 0, 1000	101 row(s) returned
✓ 2	14:16:00	SELECT COUNT(*) AS total_company FROM company LIMIT 0, 1000	1 row(s) returned
✓ 3	14:16:31	SELECT COUNT(*) AS total_transaction FROM transaction LIMIT 0, 1000	1 row(s) returned

La base de datos queda de la siguiente manera:



La base de datos consta de dos tablas:

Transaction

ID como llave primaria, mostrado con una llave amarilla

Credit_card_id y **user_id** que guardan la información de la tarjeta y el usuario que compró

Company_id que es la llave foránea, se ve con un rombo rojo. Sirve para conectar la tabla con la tabla company

Latitud y Longitud dando las coordenadas de donde se ha realizado la transacción

Timestamp indicando la fecha y hora de la transacción

Amount que indica el monto de la transacción con dos decimales

Declined con un booleano que indica si la transacción ha sido rechazada o no;

Company

ID que es llave primaria de la tabla, identificando de forma única a la empresa

Company_name, phone, email, country, website que ofrece datos básicos de contacto de la empresa.

Ejercicio 2

2.1

```

2  -- Llistat de països que estan realitzant vendes
3
4  • SELECT DISTINCT c.country FROM company c
5     INNER JOIN transaction t
6     ON c.id=t.company_id
7     WHERE t.declined = 0
8     AND c.country IS NOT NULL
9     ORDER BY c.country;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

country
Australia
Belgium
Canada
China
France
Germany
Ireland
Italy
Netherlands
New Zealand

Result 18 x

Output

Action Output

#	Time	Action	Message
✓ 2	14:16:00	SELECT COUNT(*) AS total_company FROM company LIMIT 0, 1000	1 row(s) returned
✓ 3	14:16:31	SELECT COUNT(*) AS total_transaction FROM transaction LIMIT 0, 1000	1 row(s) returned
✓ 4	14:28:35	SELECT * FROM transactions.company LIMIT 0, 1000	101 row(s) returned
✓ 5	14:28:45	SELECT DISTINCT c.country FROM transactions.company c INNER JOIN transactions.transaction t ON c.id=t.c...	16 row(s) returned
✓ 6	14:30:24	SELECT DISTINCT c.country FROM transactions.company c INNER JOIN transactions.transaction t ON c.id=t.c...	15 row(s) returned
✓ 7	14:31:25	SELECT c.country FROM transactions.company c INNER JOIN transactions.transaction t ON c.id=t.company_id...	1000 row(s) returned
✓ 8	14:31:45	SELECT DISTINCT c.country FROM transactions.company c INNER JOIN transactions.transaction t ON c.id=t.c...	15 row(s) returned
✓ 9	14:32:36	SELECT DISTINCT c.country FROM company c INNER JOIN transaction t ON c.id=t.company_id WHERE t.de...	15 row(s) returned

La consulta selecciona el país con un distinct para que cada país lo cuente solo una vez. Unimos la tabla transaction y las unimos con la palabra reservada ON. Hago una condición con where, en el que busco que las transacciones no hayan sido rechazadas y donde el país no sea nulo. Ordeno la consulta por país y me devuelve una lista de 15 países.

2.2

```

2  -- Des de quants països es generen les vendes
3
4  • SELECT COUNT(DISTINCT c.country) AS païses_ventas FROM company c
5  INNER JOIN transaction t
6  ON c.id=t.company_id
7  WHERE t.declined = 0;

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
païses_ventas			
15			

#	Time	Action	Message
5	14:28:45	SELECT DISTINCT c.country FROM transactions.company c INNER JOIN transactions.transaction t ON c.id=t.c...	16 row(s) returned
6	14:30:24	SELECT DISTINCT c.country FROM transactions.company c INNER JOIN transactions.transaction t ON c.id=t.c...	15 row(s) returned
7	14:31:25	SELECT c.country FROM transactions.company c INNER JOIN transactions.transaction t ON c.id=t.company_id...	1000 row(s) returned
8	14:31:45	SELECT DISTINCT c.country FROM transactions.company c INNER JOIN transactions.transaction t ON c.id=t.c...	15 row(s) returned
9	14:32:36	SELECT DISTINCT c.country FROM company c INNER JOIN transaction t ON c.id=t.company_id WHERE t.de...	15 row(s) returned
10	14:35:38	SELECT COUNT(DISTINCT c.country) FROM transactions.company c INNER JOIN transactions.transaction t O...	1 row(s) returned
11	14:35:55	SELECT COUNT(c.country) FROM transactions.company c INNER JOIN transactions.transaction t ON c.id=t.co...	1 row(s) returned
12	14:36:11	SELECT COUNT(DISTINCT c.country) AS païses_ventas FROM transactions.company c INNER JOIN transacti...	1 row(s) returned

Es una consulta muy similar a la anterior, pero en este caso uso el COUNT() para que me devuelva un número. Uno las tablas mediante ON y en la cláusula WHERE busco que seleccione solo las transacciones realizadas. Me devuelve 1 columna de 15 países.

2.3

```

2  -- Identifica la companyia amb la mitjana més gran de vendes.
3
4  • SELECT c.company_name, ROUND(AVG(t.amount),2) AS mitja_superior_vendes FROM company c
5     INNER JOIN transaction t
6     ON c.id=t.company_id
7     WHERE t.declined = 0
8     GROUP BY c.company_name
9     ORDER BY mitja_superior_vendes DESC
10    LIMIT 1;

```

company_name	mitja_superior_vendes
Ac Fermentum Incorporated	284.91

#	Time	Action	Message
✓ 6	14:30:24	SELECT DISTINCT c.country FROM transactions.company c INNER JOIN transactions.transaction t ON c.id=t.c...	15 row(s) returned
✓ 7	14:31:25	SELECT c.country FROM transactions.company c INNER JOIN transactions.transaction t ON c.id=t.company_id...	1000 row(s) returned
✓ 8	14:31:45	SELECT DISTINCT c.country FROM transactions.company c INNER JOIN transactions.transaction t ON c.id=t.c...	15 row(s) returned
✓ 9	14:32:36	SELECT DISTINCT c.country FROM company c INNER JOIN transaction t ON c.id=t.company_id WHERE t.de...	15 row(s) returned
✓ 10	14:35:38	SELECT COUNT(DISTINCT c.country) FROM transactions.company c INNER JOIN transactions.transaction t O...	1 row(s) returned
✓ 11	14:35:55	SELECT COUNT(c.country) FROM transactions.company c INNER JOIN transactions.transaction t ON c.id=t.co...	1 row(s) returned
✓ 12	14:36:11	SELECT COUNT(DISTINCT c.country) AS paises_ventas FROM transactions.company c INNER JOIN transacti...	1 row(s) returned
✓ 13	14:40:16	SELECT c.company_name, ROUND(AVG(t.amount),2) AS mitja_superior_vendes FROM transactions.company c...	1 row(s) returned

Selecciono el company_name, busco el average de amount y redondeo a 2 cifras. Uno las tablas, pongo una condición en que las transacciones no estén rechazadas, agrupo por el nombre de la compañía, ordeno por la media superior de ventas y limito a 1 para que me de solo un nombre.

Ejercicio 3

3.1

```

2  -- -- Mostra totes les transaccions realitzades per empreses d'Alemanya
3  • SELECT * FROM transaction
4  WHERE EXISTS (
5      SELECT id
6      FROM company
7      WHERE country='Germany'
8  ) AND declined=0;

```

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
00043A49-2949-494B-A5DD-A5BAE3B819DD	Cc5-9294	b-2458	4713	46.1999	1.43554	2024-08-28 07:16:46	395.43	0
000447FE-8650-4DCF-85DE-C7ED0EE1CAAD	Cc5-5019	b-2370	438	41.5972	12.2218	2016-12-21 20:07:18	155.63	0
00045D68-ED2E-4F2F-8186-CEE074D875D0	Cc5-6699	b-2390	2118	29.7573	-95.3796	2020-07-14 15:37:45	326.01	0
000481C3-1C26-4FEF-83A0-4CDOEB0048BD	Cc5-6696	b-2230	2115	53.5489	-113.503	2017-09-04 19:44:53	161.60	0
00051AA4-9CBE-4268-B070-C38062A1B3E2	Cc5-7606	b-2266	3025	52.2084	5.69081	2017-01-05 18:19:25	148.91	0
0008A312-EDFE-4A4F-8C99-E9C92EC3CA4D	CdJ-3358	b-2598	215	53.5535	-113.499	2023-09-23 04:51:43	294.59	0

transaction 15 x

Output

Action Output

#	Time	Action	Message
8	14:31:45	SELECT DISTINCT c.country FROM transactions.company c INNER JOIN transactions.transaction t ON c.id=t.c...	15 row(s) returned
9	14:32:36	SELECT DISTINCT c.country FROM company c INNER JOIN transaction t ON c.id=t.company_id WHERE t.de...	15 row(s) returned
10	14:35:38	SELECT COUNT(DISTINCT c.country) FROM transactions.company c INNER JOIN transactions.transaction t O...	1 row(s) returned
11	14:35:55	SELECT COUNT(c.country) FROM transactions.company c INNER JOIN transactions.transaction t ON c.id=t.co...	1 row(s) returned
12	14:36:11	SELECT COUNT(DISTINCT c.country) AS paises_ventas FROM transactions.company c INNER JOIN transacti...	1 row(s) returned
13	14:40:16	SELECT c.company_name, ROUND(AVG(t.amount),2) AS mitja_superior_vendes FROM transactions.company c...	1 row(s) returned
14	14:50:10	SELECT * FROM transaction WHERE EXISTS (SELECT id FROM company WHERE...	1000 row(s) returned
15	15:00:13	SELECT * FROM company c WHERE NOT EXISTS (SELECT company_id FROM transaction t ...	0 row(s) returned
16	15:00:19	SELECT company_name FROM company WHERE EXISTS (SELECT company_id FROM transaction ...	101 row(s) returned
17	15:00:34	SELECT * FROM transaction WHERE EXISTS (SELECT id FROM company WHERE...	1000 row(s) returned
18	19:06:29	SELECT * FROM transaction WHERE EXISTS (SELECT id FROM company WHERE...	1000 row(s) returned

Selecciono todo de la tabla transaction y uso la cláusula WHERE EXISTS y donde los pedidos no hayan sido rechazados, con una subconsulta que seleccionando el id de la tabla company, y la restricción WHERE country='Germany', el resultado muestra 1000 columnas.

3.2

```

12  -- Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions.
13
14  • SELECT company_name FROM company
15  WHERE EXISTS (
16      SELECT company_id
17      FROM transaction
18      WHERE amount > (
19          SELECT ROUND(AVG(amount),2)
20          FROM transaction));

```

company_name
Ac Fermentum Incorporated
Magna A Neque Industries
Fusce Corp.
Convallis In Incorporated
Ante Taculis Nec Foundation
Donec Ltd
Sed Nunc Ltd
Amet Nulla Donec Corporation
Nascetur Ridiculus Mus Inc.

company 20 x

Output

Action Output

#	Time	Action	Message
18	19:06:29	SELECT * FROM transaction WHERE EXISTS (SELECT id FROM company WHERE...	1000 row(s) returned
19	19:13:12	SELECT company_name FROM company WHERE EXISTS (SELECT company_id FROM transaction ...	101 row(s) returned
20	19:14:18	(SELECT AVG(amount) FROM transaction))	Error Code: 1064. You have an error in your SQL
21	19:14:29	(SELECT AVG(amount) FROM transaction)	1 row(s) returned
22	19:14:49	(SELECT ROUND(AVG(amount),2) FROM transaction)	1 row(s) returned
23	19:15:00	(SELECT company_id FROM transaction WHERE amount > (SELECT ROUND(AVG(amount),2...	46722 row(s) returned
24	19:15:09	SELECT company_name FROM company WHERE EXISTS (SELECT company_id FROM transaction ...	101 row(s) returned

En esta consulta, selecciono el company_name de la tabla company y la añado la restricción WHERE EXISTS, añado una subconsulta que busca el company_id de la tabla transacciones y la restricción es el amount, que debe ser mayor a la consulta anidada que busca el average de amount, y lo redondeo a dos decimales. El resultado son 101 filas.

3.3

```
--
-- Eliminaran del sistema les empreses que no tenen transaccions registrades, entrega el llistat d'aquestes empreses.
23 • SELECT * FROM company c
24 WHERE NOT EXISTS (
25     SELECT company_id
26     FROM transaction t
27     WHERE c.id=t.company_id
28 );
```

id	company_name	phone	email	country	website
1	company_name	phone	email	country	website

company 21 x

Output

Action Output

#	Time	Action	Message
19	19:13:12	SELECT company_name FROM company WHERE EXISTS (SELECT company_id FROM transaction ...	101 row(s) returned
20	19:14:18	(SELECT AVG(amount) FROM transaction))	Error Code: 1064. You have an error in your SQL sy
21	19:14:29	(SELECT AVG(amount) FROM transaction)	1 row(s) returned
22	19:14:49	(SELECT ROUND(AVG(amount),2) FROM transaction)	1 row(s) returned
23	19:15:00	(SELECT company_id FROM transaction WHERE amount > (SELECT ROUND(AVG(amount),2...	46722 row(s) returned
24	19:15:09	SELECT company_name FROM company WHERE EXISTS (SELECT company_id FROM transaction ...	101 row(s) returned
25	19:17:49	SELECT * FROM company c WHERE NOT EXISTS (SELECT company_id FROM transaction t ...	0 row(s) returned

En esta consulta, busco todo de la tabla company y añado la restricción WHERE NOT EXISTS, con una subconsulta que busca el company_id de la tabla transaction y una restricción con WHERE que une las llaves primaria y foránea de las tablas. La consulta devuelve 0 columnas ya que no hay ninguna empresa sin transacciones.

Ejercicio 4

Creo la tabla credit_cards con los atributos solicitados:

```
• CREATE TABLE IF NOT EXISTS credit_card(
    id VARCHAR(15) PRIMARY KEY,
    iban VARCHAR(255),
    pan VARCHAR(255),
    pin VARCHAR(10),
    cvv VARCHAR(4),
    expiring_date VARCHAR(10)
);
```

Los atributos son un ID que es llave primaria, que será la llave foránea de transaction para conectar ambas tablas. El resto de datos son los identificadores de cada tarjeta (iban, pan, pin, cvv, expiring_date)

Introduzco los datos en la tabla con un INSERT INTO credit_card VALUES, tal y como se ve en la imagen:

```
--
743 -- Insertamos datos de credit_card
744 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2938', 'TR301950312213576817638661', '542
745 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2945', 'D026854763748537475216568689', '5
746 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2952', 'BG45IVQL52710525608255', '4556 45
747 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2959', 'CR7242477244335841535', '37246137
748 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2966', 'BG72LKTQ15627628377363', '448566
749 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2973', 'PT87806228135092429456346', '544
750 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2980', 'DE39241881883086277136', '402400
751 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2987', 'GE89681434837748781813', '3763 74
752 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2994', 'BH62714428368066765294', '3442832
753 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3001', 'CY49087426654774581266832110', '5
754 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3008', 'LU507216693616119230', '448574446
755 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3015', 'PS119398216295715968342456821', '
756 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3022', 'GT91695162850556977423121857', '5
757 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3029', 'AZ62317413982441418123739746', '3
758 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3036', 'AZ39336002925842865843941994', '3
759 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3043', 'TN6488143310514852179535', '45567
760 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3050', 'FR5167744369175836831854477', '40
761 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3057', 'LU931822574697545215', '3484 6217
762 • INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3064', 'DE146065545440753377677773133', '
C
```

Una vez hecho, reviso que se hayan insertado los datos:

```
1 • SELECT * FROM transactions.credit_card;
2 • SELECT COUNT(*) FROM credit_card;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [↗](#)

	COUNT(*)
▶	5001

Result 2 x [Read C](#)

Output

Action Output

#	Time	Action	Message
✓	21 19:14:29	(SELECT AVG(amount) FROM transaction)	1 row(s) returned
✓	22 19:14:49	(SELECT ROUND(AVG(amount),2) FROM transaction)	1 row(s) returned
✓	23 19:15:00	(SELECT company_id FROM transaction WHERE amount > (SELECT ROUND(AVG(amount),2) FROM transaction))	46722 row(s) returned
✓	24 19:15:09	SELECT company_name FROM company WHERE EXISTS (SELECT company_id FROM transaction WHERE company_id = company_id)	101 row(s) returned
✓	25 19:17:49	SELECT * FROM company c WHERE NOT EXISTS (SELECT company_id FROM transaction t WHERE c.company_id = t.company_id)	0 row(s) returned
✓	26 19:25:27	SELECT * FROM transactions.credit_card LIMIT 0, 1000	1000 row(s) returned
✓	27 19:25:48	SELECT COUNT(*) FROM credit_card LIMIT 0, 1000	1 row(s) returned

Se han añadido 5001 registros. A continuación, altero la tabla transaction para poder crear la llave foránea, que conectará credit_card con transaction:


```

4
5 • ALTER TABLE transaction
6 ADD CONSTRAINT fk_card_transaction
7 FOREIGN KEY (credit_card_id) REFERENCES credit_card(id)
8 ON UPDATE CASCADE;
9

```

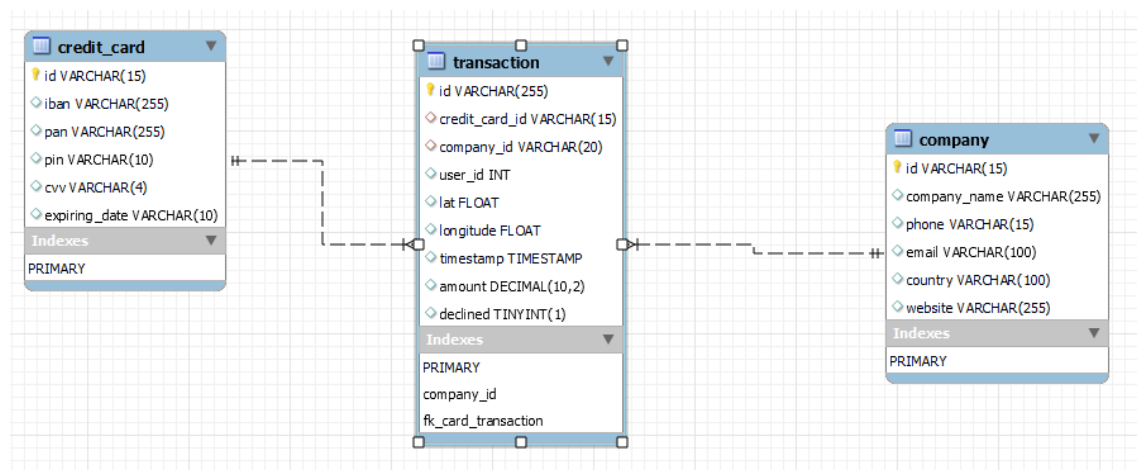
Añado el constraint `fk_card_transaction` que es la foreign key `credit_card_id` que referencia a la tabla `credit_card(id)`. Finalmente, añado `on update cascade` para que, en caso de modificar o actualizar algún dato en la tabla madre (`transaction`), se actualice automáticamente en la tabla hija.

```

DDL for transactions.transaction
1 CREATE TABLE `transaction` (
2   `id` varchar(255) NOT NULL,
3   `credit_card_id` varchar(15) DEFAULT NULL,
4   `company_id` varchar(20) DEFAULT NULL,
5   `user_id` int DEFAULT NULL,
6   `lat` float DEFAULT NULL,
7   `longitude` float DEFAULT NULL,
8   `timestamp` timestamp NULL DEFAULT NULL,
9   `amount` decimal(10,2) DEFAULT NULL,
10  `declined` tinyint(1) DEFAULT NULL,
11  PRIMARY KEY (`id`),
12  KEY `company_id` (`company_id`),
13  KEY `fk_card_transaction` (`credit_card_id`),
14  CONSTRAINT `fk_card_transaction` FOREIGN KEY (`credit_card_id`) REFERENCES `credit_card` (`id`) ON UPDATE CASCADE,
15  CONSTRAINT `transaction_ibfk_1` FOREIGN KEY (`company_id`) REFERENCES `company` (`id`)
16 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

```

Reviso la tabla y veo que la llave foránea se ha creado correctamente.



Esta nueva relación es de 1:n. En la tabla `credit_card`, vemos que las dos líneas significan mínimo 1 y máximo 1, lo que implica que una tarjeta es única, no puede tener los mismos datos que las otras tarjetas. En el lado de `transaction`, vemos que se ha añadido al atributo `company_id` un rombo rojo, ya que la hemos convertido en la llave foránea de `credit_card`. Vemos en su extremo izquierdo que puede ser 0 o muchas. Una tarjeta puede hacer varias transacciones, pero una transacción, solo puede pertenecer a 1 tarjeta.

Ejercicio 5

```
2 SELECT * FROM credit_card WHERE id = 'CcU-2938';
3 UPDATE credit_card
4 SET iban = 'TR323456312213576817699999'
5 WHERE id = 'CcU-2938';
```

5011 19:35:16 UPDATE credit_card SET iban = 'TR323456312213576817699999' WHERE id = 'CcU-2938' 1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0 0.031 sec

En esta pregunta, solicitaban modificar el iban según un id concreto. En primer lugar, hago un select all de la tabla credit_card y añado la restricción id solicitado para comprobar que dicho id existe. Una vez hecho, uso el Update credit_card, añado el iban con el número solicitado usando el comando SET iban='TR323456312213576817699999' con una última restricción que dice dónde añadirlo (en el id='CcU-2938').

Finalmente muestro de nuevo la consulta y veo que se ha modificado correctamente.

```
6 SELECT * FROM credit_card WHERE id = 'CcU-2938';
```

Result Grid						
Filter Rows:						
Edit: 						
Export/Import:  						
Wrap						
id	iban	pan	pin	cvv	expiring_date	
CcU-2938	TR323456312213576817699999	5424465566813633	3257	984	10/30/22	
NULL	NULL	NULL	NULL	NULL	NULL	

Ejercicio 6

```
INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined)
VALUES ('10881D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', '9999', '829.999', '-117.999', '111.11', '0');
```

Añado en la tabla transaction los Nuevos datos de una transacción. Sin embargo, el dato timestamp no lo proporciona, pero debido a que la tabla permite nulos en el atributo timestamp, lo elimino y lo dejo en blanco, por tanto el sistema lo dejará en null.

Al hacer run en el código, me salta “error code 1452”. Fallo con la foreign key constraint. Los datos que dan problema son precisamente credit_card_id y company_id. Ambas son fk de las tablas credit_card y company. Por tanto, una forma de solventarlo es insertar ambos datos en sus respectivas tablas (en company añado el valor id, y en credit_card añado el valor id).

```
INSERT INTO company (id) VALUES ('b-9999');
INSERT INTO credit_card (id) VALUES ('CcU-9999');
```

Vuelvo a lanzar la transacción, y esta vez la acepta:

```
1 • SELECT * FROM transactions.transaction;
2 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined)
3 • VALUES ('10881D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', '9999', '829.999', '-117.999', '111.11', '0');
4 • INSERT INTO company (id) VALUES ('b-9999');
5 • INSERT INTO credit_card (id) VALUES ('CcU-9999');
```

Output

Action Output

#	Time	Action	Message
5020	20:23:42	INSERT INTO company (id) VALUES (b-9999)	1 row(s) affected
5021	20:23:42	INSERT INTO credit_card (id) VALUES (CcU-9999)	1 row(s) affected
5022	20:24:43	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined) VALUES (1...	1 row(s) affected

```
6 • SELECT * FROM transaction WHERE credit_card_id = 'CcU-9999';
```

Result Grid

Filter Rows:

Edit: Export/Import: Wrap Cell Content:

	id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
▶	10881D1D-5B23-A76C-55EF-C568E49A99DD	CcU-9999	b-9999	9999	829.999	-117.999	NULL	111.11	0
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Ejercicio 7

```

8 • ALTER TABLE credit_card
9   DROP COLUMN pan;
10 • SELECT * FROM credit_card;

```

Result Grid | Filter Rows: | Edit: | Export/

	id	iban	pin	cvv	expiring_date
▶	CcS-4857	XX4857591835292505850771	1819	467	09/27/25
	CcS-4858	XX8581768137002436094025	3964	817	12/28/28
	CcS-4859	XX7826930491423553609370	4983	277	11/26/26
	CcS-4860	XX5559590368835304645299	6876	661	07/27/27
	CcS-4861	XX2035182877195191627307	5710	398	04/25/26
	CcS-4862	XX4774721462463645409758	4042	174	11/27/26
	CcS-4863	XX1476829664245046207111	5969	449	12/27/29
	CcS-4864	XX0000000000000000000000	0000	000	00/00/00

credit_card 6 x

Uso el comando alter table credit_card, y le añado drop column pan. Por último, muestro por pantalla que la transacción se haya realizado con éxito.

Ejercicio 8

En primer lugar, creo la base de datos ejercicios_csv, donde añadiré las tablas y usaré los archivos csv para cargar los datos.

```

1 • CREATE DATABASE IF NOT EXISTS ejercicios_csv;
2 • USE ejercicios_csv;

```

A continuación, creo las tablas:

Transaction:

The screenshot shows a SQL IDE with a query editor and an output pane. The query editor contains a `CREATE TABLE IF NOT EXISTS transaction` statement with the following attributes: `id VARCHAR(100) PRIMARY KEY`, `card_id VARCHAR(50)`, `business_id VARCHAR(15)`, `timestamp timestamp`, `amount DECIMAL(10,2)`, `declined BOOLEAN NOT NULL DEFAULT 0`, `product_ids VARCHAR(255) NOT NULL`, `user_id INT NOT NULL`, `lat FLOAT DEFAULT NULL`, `longitude FLOAT DEFAULT NULL`, `discount_amount DECIMAL(10,2)`, `tax_amount DECIMAL(10,2)`, `shipping_amount DECIMAL(10,2)`, `channel VARCHAR(50)`, `campaign_id VARCHAR(50)`, `device_type VARCHAR(20)`, `is_international BOOLEAN NOT NULL DEFAULT 0`, `decline_reason VARCHAR(150)`, and `distance_km DECIMAL(10,2)`. The output pane shows the execution of this statement and several `SELECT` queries. The final message for the `CREATE TABLE` statement is "0 row(s) affected, 1 warning(s): 1050 Table 'transaction' already exists".

```

5 CREATE TABLE IF NOT EXISTS transaction(
6   id VARCHAR(100) PRIMARY KEY,
7   card_id VARCHAR(50),
8   business_id VARCHAR(15),
9   timestamp timestamp,
10  amount DECIMAL(10,2),
11  declined BOOLEAN NOT NULL DEFAULT 0,
12  product_ids VARCHAR(255) NOT NULL,
13  user_id INT NOT NULL,
14  lat FLOAT DEFAULT NULL,
15  longitude FLOAT DEFAULT NULL,
16  discount_amount DECIMAL(10,2),
17  tax_amount DECIMAL(10,2),
18  shipping_amount DECIMAL(10,2),
19  channel VARCHAR(50),
20  campaign_id VARCHAR(50),
21  device_type VARCHAR(20),
22  is_international BOOLEAN NOT NULL DEFAULT 0,
23  decline_reason VARCHAR(150),
24  distance_km DECIMAL(10,2)
25 );

```

#	Time	Action	Message
26	19:25:27	SELECT * FROM transactions.credit_card LIMIT 0, 1000	1000 row(s) returned
27	19:25:48	SELECT COUNT(*) FROM credit_card LIMIT 0, 1000	1 row(s) returned
28	19:38:34	SELECT * FROM transactions.credit_card LIMIT 0, 1000	1000 row(s) returned
29	19:39:18	SELECT * FROM credit_card WHERE id = 'CcU-2938' LIMIT 0, 1000	1 row(s) returned
30	19:39:25	SELECT * FROM credit_card WHERE id = 'CcU-2938' LIMIT 0, 1000	1 row(s) returned
31	19:47:08	SELECT * FROM transaction WHERE credit_card_id = 'CcU-9999' LIMIT 0, 1000	1 row(s) returned
32	19:57:47	CREATE TABLE IF NOT EXISTS transaction(id VARCHAR(100) PRIMARY KEY, card_id VARCHAR(50), busin...	0 row(s) affected, 1 warning(s): 1050 Table 'transaction' already exists

Creo la tabla con `create table if not exists transaction` con los siguientes atributos:

`Id` que será la llave primaria, y otros datos relacionados con la transacción como `card_id`, `business_id`, `timestamp` (que dirá fecha y hora de la transacción), `amount`, un booleano de rechazado, `product_id`, `user_id`, latitud y longitud (lugar de la transacción), monto de descuento, la tasa de impuestos, el envío, canal, la campaña, dispositivo desde donde se realiza, booleano de si es internacional o nacional, razón de rechazo, y la distancia en km.

La siguiente tabla que creo es la de usuarios, que al revisar los archivos csv, veo que los campos son los mismos, por lo que decido crear una sola tabla para ambos archivos (usuarios europeos y americanos).

```

7  CREATE TABLE IF NOT EXISTS users(
8      id INT PRIMARY KEY,
9      name VARCHAR(100) NOT NULL,
0      surname VARCHAR(100) NOT NULL,
1      phone VARCHAR(50),
2      email VARCHAR(100) NOT NULL,
3      birth_date VARCHAR(50),
4      country VARCHAR (50) NOT NULL,
5      city VARCHAR (50) NOT NULL,
6      postal_code VARCHAR (20) NOT NULL,
7      address VARCHAR (150) NOT NULL,
8      signup_date VARCHAR(50) NOT NULL,
9      user_segment VARCHAR (50) NOT NULL,
0      income_band VARCHAR (20)
1  );

```

Hago un ID que será la llave primaria, y otros datos de contacto, como el nombre y apellidos, teléfono, mail, fecha de nacimiento, país, ciudad, código postal, dirección, fecha de alta, segmento. Altero la tabla users para crear una columna que sea continente. Actualizo la tabla y añado un case-else en el que establezco en la columna continente que si el país se encuentra en 'United States' o 'Canada' añada 'America', y 'Europe' al resto.

```

83  ALTER TABLE users
84  ADD continent VARCHAR(50);
85  UPDATE users
86  SET continent = CASE
87      WHEN country IN ('United States', 'Canada') THEN 'America'
88      ELSE 'Europe'
89  END;
90

```

Output

#	Time	Action	Message
✓ 35	20:10:17	SELECT * FROM ejercicios_csv.companies LIMIT 0, 1000	100 row(s) returned
✓ 36	20:20:36	USE ejercicios_csv	0 row(s) affected
✓ 37	20:22:57	ALTER TABLE users ADD continent VARCHAR(50)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
✗ 38	20:22:57	UPDATE company SET continent = CASE WHEN country IN ('United States', 'Canada') THEN 'America' ELSE...	Error Code: 1146. Table 'ejercicios_csv.company' doesn't exist
✗ 39	20:23:27	ALTER TABLE users ADD continent VARCHAR(50)	Error Code: 1060. Duplicate column name 'continent'
✓ 40	20:23:34	UPDATE users SET continent = CASE WHEN country IN ('United States', 'Canada') THEN 'America' ELSE 'E...	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0
✓ 41	20:23:42	SELECT * FROM ejercicios_csv.users LIMIT 0, 1000	1000 row(s) returned

```
1 • SELECT * FROM ejercicios_csv.users;
```

	birth_date	country	city	postal_code	address	signup_date	user_segment	income_band	continent
	Dec 3, 1981	United States	San Antonio	78201	P.O. Box 455	2015-03-14	young_professional	medium	America
ot...	May 11, 1998	United States	Houston	77001	491 Neque St.	2015-03-25	frequent_shopper	high	America
	Apr 3, 1993	United States	New York	10001	P.O. Box 191	2013-06-14	family	low	America
	May 17, 1981	United States	Phoenix	85001	Ap #644-7695 Nec	2015-04-03	frequent_shopper	high	America
et	Jun 20, 1980	United States	Los Angeles	90001	745-3814 Metus. Rd.	2015-02-14	young_professional	high	America
	May 27, 1991	United States	Chicago	60601	Ap #558-7250 Vivamus Rd.	2014-09-16	young_professional	medium	America
edu	Nov 28, 1995	United States	Phoenix	85001	5363 Rhoncus Street	2014-12-01	young_professional	high	America
	Feb 19, 1983	United States	San Antonio	78201	772-6878 Sociosqu Av.	2014-12-10	occasional	medium	America
	Dec 22, 1994	United States	San Diego	92101	5438 Odio. Avenue	2014-01-21	occasional	medium	America
	Oct 14, 1980	United States	Los Angeles	90001	465-230 Ullamcorper	2014-06-16	family	medium	America

```
3 • CREATE TABLE IF NOT EXISTS companies(
4     company_id VARCHAR(15) PRIMARY KEY,
5     company_name VARCHAR (255) NOT NULL,
6     phone VARCHAR (20),
7     email VARCHAR (100) NOT NULL,
8     country VARCHAR (50) NOT NULL,
9     website VARCHAR (100) NOT NULL,
10    merchant_category VARCHAR (50) NOT NULL,
11    merchant_price_position VARCHAR (50) NOT NULL
12 );
13
```

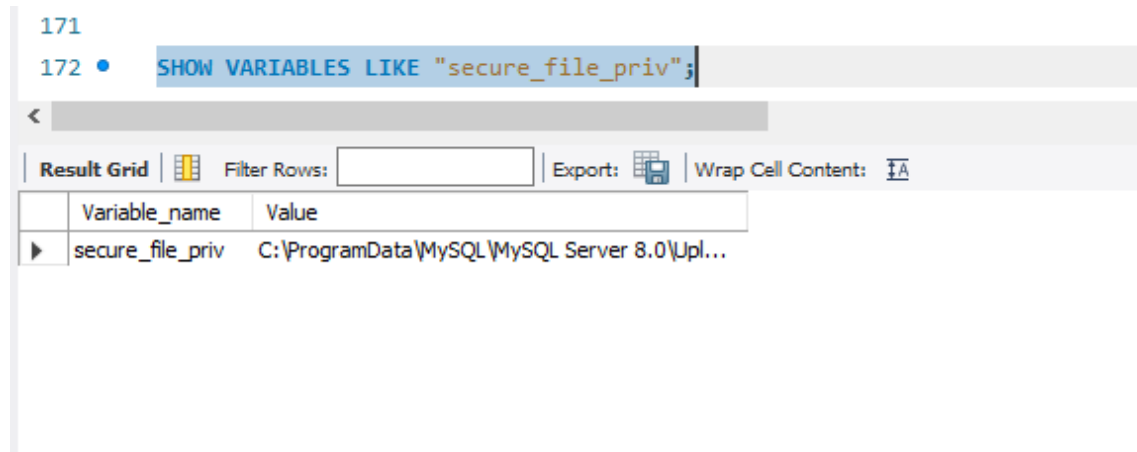
La siguiente tabla que creo es companies, con un company_id que hace de llave primaria, y otros datos de la empresa que realiza las compras, como el company_name, teléfono, mail, país, web , categoría.

```
95 • CREATE TABLE IF NOT EXISTS credit_cards(
96     id VARCHAR(50) PRIMARY KEY,
97     user_id INT NOT NULL,
98     iban VARCHAR(100) NOT NULL,
99     pan VARCHAR(20) NOT NULL,
100    pin VARCHAR(6) NOT NULL,
101    cvv VARCHAR(4) NOT NULL,
102    track1 VARCHAR(100),
103    track2 VARCHAR(100),
104    expiring_date DATE,
105    card_type VARCHAR(30) NOT NULL,
106    card_renewal_flag BOOLEAN NOT NULL DEFAULT 0
107 );
```

Por último, creo la tala credit_cards, que almacena los datos de las tarjetas de crédito. Un ID hace de llave primaria, y el resto son datos identificadores de cada tarjeta como el user_id, iban, pan, pin, cvv, track1, track2, expiring_date, card_type y un booleano para las renovaciones.

El cvv uso un varchar de 4, ya que en algunos países usan un código de 4 dígitos, aunque la mayoría de países es de 4.

Es momento de cargar los datos en las tablas, para ello, he de usar los documentos CSV que nos han sido proporcionados. Los introduzco en la carpeta ProgramData para que MySQL Workbench los pueda leer. Veo que la ruta es la indicada en la consulta.



Una vez añadidos, empiezo a cargar datos:

```
--
118 • LOAD DATA
119 INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__credit_cards.csv'
120 INTO TABLE credit_cards
121 FIELDS TERMINATED BY ','
122 IGNORE 1 LINES
123 (id, user_id, iban, pan, pin, cvv, track1, track2, @var_fecha, card_type, card_renewal_flag)
124 SET expiring_date = STR_TO_DATE (@var_fecha, '%m/%d/%y');
```

Para la tabla credit_cards uso LOAD DATA INFILE con la ruta al documento .csv y lo introduzco en la tabla credit_cards, con los campos separados con ',' según el archivo y pido que ignore las cabeceras del documento, con IGNORE 1 LINES.

Debido a que la fecha está en formato mes/día/año, y MySQL Workbench es muy estricto con las fechas, creo una variable temporal con @var_fecha y le paso la función STR_TO_DATE, pasándole el parámetro de la fecha que recogerá del archivo csv, y cómo lo tiene que leer (mes, día, año).

```
--
6 • LOAD DATA
7 INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__companies.csv'
8 INTO TABLE companies
9 FIELDS TERMINATED BY ','
0 IGNORE 1 LINES;
```

En este caso, añado los datos del archivo companies en la tabla companies, le indico que los campos acaban en ',' y que ignore la primera fila.

En cuanto a la tabla users, debido a que el archivo original estaba dividido en dos distintos, he de hacer dos cargas diferentes:

```
33 • LOAD DATA
34 INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__american_users.csv'
35 INTO TABLE users
36 FIELDS TERMINATED BY ','
37 OPTIONALLY ENCLOSED BY '"'
38 LINES TERMINATED BY '\n'
39 IGNORE 1 LINES;
40
41 • LOAD DATA
42 INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__european_users.csv'
43 INTO TABLE users
44 FIELDS TERMINATED BY ','
45 OPTIONALLY ENCLOSED BY '"'
46 LINES TERMINATED BY '\n'
47 IGNORE 1 LINES;
```

En este caso, tuve problemas para cargar los datos, ya que contenía saltos de línea estándar (muy común en Linux y MacOS). El FIELDS TERMINATED BY ',' implica que los campos están separados por coma, OPTIONALLY ENCLOSED BY '"' indica que algunos datos están entre comillas, y se advierte para que no lo confunda con otras columnas. Y por último LINES TERMINATED BY '\n', que este es el salto de línea, cada fila acaba con nueva línea o line feed. Por último ignoro las cabeceras.

```
• LOAD DATA
  INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__transactions.csv'
  INTO TABLE transaction
  FIELDS TERMINATED BY ';'
  OPTIONALLY ENCLOSED BY '"'
  LINES TERMINATED BY '\n'
  IGNORE 1 LINES;
```

Por último, cargo los datos de la tabla transactions en la tabla transaction. Los campos terminan en punto y coma ';', OPTIONALLY ENCLOSED BY '"' para que tenga en cuenta los datos entre comillas, y LINES TERMINATED BY '\n' en que cada fila acaba con nueva línea o line feed. Por último ignoro cabeceras.

Por último, añado las llaves foráneas:

```
• ALTER TABLE transaction
  ADD CONSTRAINT fk_transaction_users
  FOREIGN KEY (user_id) REFERENCES users(id)
  ON UPDATE CASCADE;
```

Añado la constraint donde el user_id de la tabla transaction referencia a la tabla users y a la llave primaria id.

```

--
52 • ALTER TABLE transaction
53 ADD CONSTRAINT fk_transaction_companies
54 FOREIGN KEY (business_id) REFERENCES companies(company_id)
55 ON UPDATE CASCADE;

```

Añado la constraint donde el business_id de la tabla transaction referencia a la tabla companies y la llave primaria es company_id

```

--
.67 • ALTER TABLE transaction
.68 ADD CONSTRAINT fk_transaction_card
.69 FOREIGN KEY (card_id) REFERENCES credit_cards(id)
.70 ON UPDATE CASCADE;
.71

```

Añador la constraint donde el card_id de la tabla transaction referencia a la tabla credit_cards y la llave primaria es id.

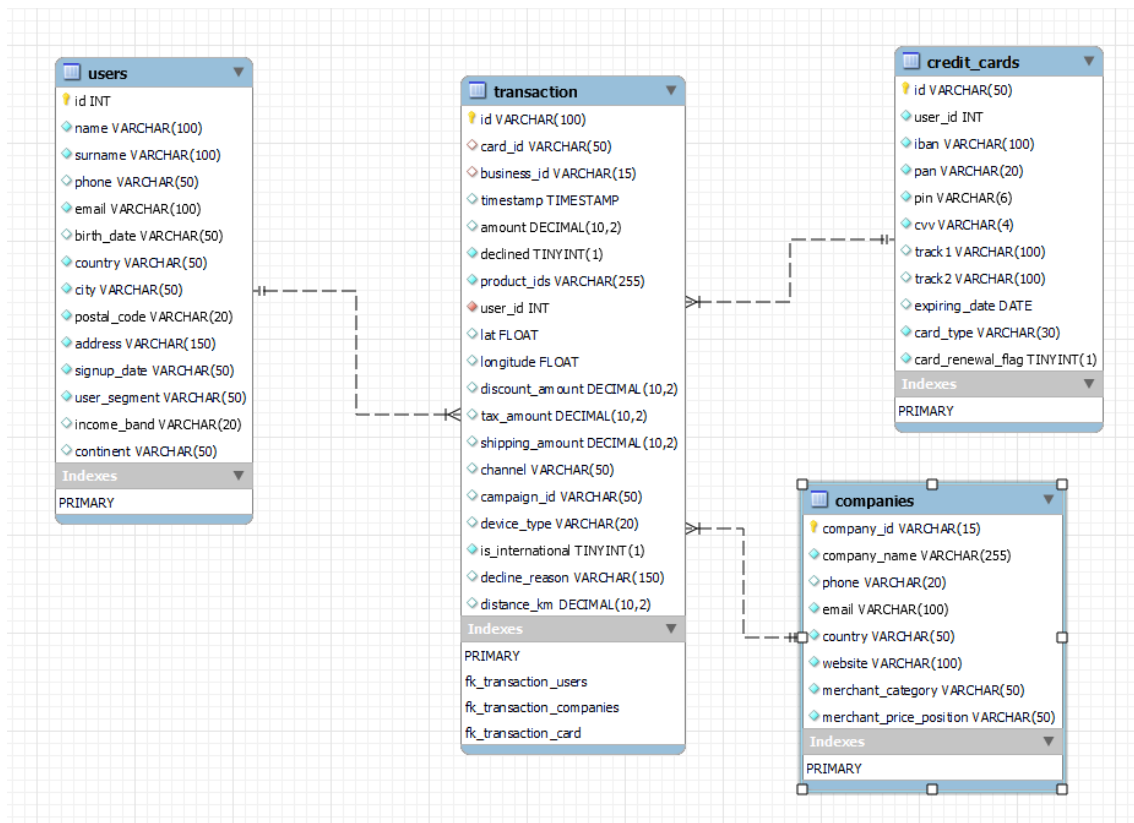
En todas ellas, añado ON UPDATE CASCADE para que, en caso de actualizar, modificar, borrar datos de la tabla madre (transaction), actualice en cascada en las hijas.

```

1 CREATE TABLE `transaction` (
2   `id` varchar(100) NOT NULL,
3   `card_id` varchar(50) DEFAULT NULL,
4   `business_id` varchar(15) DEFAULT NULL,
5   `timestamp` timestamp NULL DEFAULT NULL,
6   `amount` decimal(10,2) DEFAULT NULL,
7   `declined` tinyint(1) NOT NULL DEFAULT '0',
8   `product_ids` varchar(255) NOT NULL,
9   `user_id` int NOT NULL,
10  `lat` float DEFAULT NULL,
11  `longitude` float DEFAULT NULL,
12  `discount_amount` decimal(10,2) DEFAULT NULL,
13  `tax_amount` decimal(10,2) DEFAULT NULL,
14  `shipping_amount` decimal(10,2) DEFAULT NULL,
15  `channel` varchar(50) DEFAULT NULL,
16  `campaign_id` varchar(50) DEFAULT NULL,
17  `device_type` varchar(20) DEFAULT NULL,
18  `is_international` tinyint(1) NOT NULL DEFAULT '0',
19  `decline_reason` varchar(150) DEFAULT NULL,
20  `distance_km` decimal(10,2) DEFAULT NULL,
21  PRIMARY KEY (`id`),
22  KEY `fk_transaction_users` (`user_id`),
23  KEY `fk_transaction_companies` (`business_id`),
24  KEY `fk_transaction_card` (`card_id`),
25  CONSTRAINT `fk_transaction_card` FOREIGN KEY (`card_id`) REFERENCES `credit_cards` (`id`) ON UPDATE CASCADE,
26  CONSTRAINT `fk_transaction_companies` FOREIGN KEY (`business_id`) REFERENCES `companies` (`company_id`) ON UPDATE CASCADE,
27  CONSTRAINT `fk_transaction_users` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`) ON UPDATE CASCADE
28 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

```

Finalmente, la tabla queda así. Las llaves foráneas se han creado correctamente.



Una vez creada la base de datos y las tablas, queda este modelo en estrella, la tabla transaction sería la tabla de hechos, que registra los eventos y métricas. Aquí hemos puesto los atributos anteriormente descritos. Hemos añadido la llave primaria ID, y en la misma tabla podemos ver tres rombos rojos que son las llaves foráneas de las otras tablas (card_id, business_id, user_id). Al desplegar indexes vemos la primary key (id) y las constraints (llaves foráneas) creadas.

Las otras 3 tablas (users, companies, credit_cards) son las tablas de dimensiones, que aportan el contexto y la información a la tabla transacciones.

Las relaciones son:

Users – transaction 1:n. un usuario puede realizar varias transacciones, pero una transacción pertenece a un único usuario.

Companies – transaction 1:n una compañía puede registrar muchas transacciones, pero una transacción pertenece a una única compañía.

Credit_cards – transactions 1:n una tarjeta de crédito puede hacer varios pedidos, pero un pedido sólo puede pertenecer a una tarjeta.

Ejercicio 9

Automat
toolbar t
caret pc

```

5  -- Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.
6
7  SELECT *
8  FROM users u
9  WHERE (
10     SELECT COUNT(id) AS total_transacciones
11     FROM transaction t
12     WHERE t.user_id = u.id
13     AND declined=0
14     ) >80;

```

id	name	surname	phone	email	birth_date	country	city	postal_code	address
185	Molly	Gillam	0800 120 8023	donec@outlook.couk	Dec 21, 1993	United Kingdom	London	EC1A 1BB	P.O. Box 202, 5638 Mi
289	Dxwgi	Hwcru	+98-309-8797	dxwgi.hwcru@example.com	Aug 20, 1976	Germany	Stuttgart	70173	82 Hwcru Street
318	Bnyr	Astuw	+33-120-9644	bnyr.astuw@example.com	May 3, 1974	Italy	Genoa	16100	53 Astuw Street
454	Sfzoh	Xgvfridxs	+58-495-3945	sfzoh.xgvfridxs@example.com	Aug 28, 1962	Poland	Gdansk	80-001	52 Xgvfridxs Street

users 27 x

Output

Action Output

#	Time	Action	Message
38	20:22:57	UPDATE company SET continent = CASE WHEN country IN ('United States', 'Canada') THEN 'America' ELS...	Error Code: 1146. Table 'ejercicios_csv.company' doesn't exist
39	20:23:27	ALTER TABLE users ADD continent VARCHAR(50)	Error Code: 1060. Duplicate column name 'continent'
40	20:23:34	UPDATE users SET continent = CASE WHEN country IN ('United States', 'Canada') THEN 'America' ELSE E...	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0
41	20:23:42	SELECT * FROM ejercicios_csv.users LIMIT 0, 1000	1000 row(s) returned
42	20:28:42	SHOW VARIABLES LIKE "secure_file_priv"	1 row(s) returned
43	20:33:33	SELECT * FROM consultas_csv.credit_cards LIMIT 0, 1000	1000 row(s) returned
44	21:15:06	SELECT * FROM users u WHERE (SELECT COUNT(id) AS total_transacciones FROM transaction t WHERE t...	4 row(s) returned

En esta consulta, selecciono todo de la tabla users con la restricción WHERE y una subconsulta donde cuento el id como total transacciones de la tabla transaction, uno las dos tablas con una restricción y añado que no haya sido rechazado. Por último que sea mayor a 80. La resultado son 4 columnas.

Ejercicio 10

```

31  SELECT cc.iban, ROUND(AVG(t.amount),2) AS media_gasto
32  FROM credit_cards cc
33  INNER JOIN transaction t
34  ON cc.id=t.card_id
35  INNER JOIN companies c
36  ON t.business_id=c.company_id
37  WHERE c.company_name='Donec Ltd'
38  GROUP BY cc.iban;
39
40  -- Identifica els cinc dies que es va generar la quantitat més gran d'ingressos a l'empresa per vendes.

```

iban	media_gasto
CA901355773586771626108990	361.01
US729212642816955651428791	142.56
US899450803230933276819251	246.90
US581097432668325541142297	154.01
CA441226267105857473570574	250.51
CA332171962352073015086775	203.95
US985359628701198880628720	438.16
US788111972997693675524914	520.37

Result 29 x

Output

Action Output

#	Time	Action	Message
44	21:15:06	SELECT * FROM users u WHERE (SELECT COUNT(id) AS total_transacciones FROM transaction t WHERE t...	4 row(s) returned
45	21:20:12	SELECT u.id, u.name, u.surname, t.total_transacciones FROM users u WHERE (SELECT COUNT(id) AS total_tr...	Error Code: 1054. Unknown column 't.total_transacciones'
46	21:20:44	SELECT * FROM users u WHERE (SELECT COUNT(id) AS total_transacciones FROM transaction t WHERE t...	4 row(s) returned
47	21:21:59	SELECT credit_cards.iban, ROUND(AVG(transaction.amount),2) AS media_gasto FROM credit_cards INNER JO	371 row(s) returned

En esta consulta selecciono el iban de la tabla credit_cards, y el average amount, que redondeo a dos dígitos de la tabla transaction. Las uno con inner join y con un ON juntando claves primarias y foráneas, y añado una tercera tabla companies para poder buscar la empresa solicitada en el ejercicio. Creo una condición con WHERE el nombre de la empresa sea 'Donec Ltd' y agrupo por el iban.

Nivell 2

Ejercicio 1

```
40 -- Identifica els cinc dies que es va generar la quantitat més gran d'ingressos a l'empresa per vendes.
41 -- Mostra la data de cada transacció juntament amb el total de les vendes.
42 • SELECT DATE(timestamp) AS fechas_ventas, SUM(amount) AS total_ventas
43 FROM transaction
44 GROUP BY DATE(timestamp)
45 ORDER BY total_ventas DESC
46 LIMIT 5;
```

fechas_ventas	total_ventas
2019-11-18	15840.24
2023-02-20	15728.40
2022-12-13	15129.20
2019-03-18	14896.47
2017-12-20	14367.96

Result 31 x

Output

#	Time	Action	Message
✓	46 21:20:44	SELECT * FROM users u WHERE (SELECT COUNT(id) AS total_transacciones FROM transaction t WHERE t....	4 row(s) returned
✓	47 21:21:59	SELECT credit_cards.iban, ROUND(AVG(transaction.amount),2) AS media_gasto FROM credit_cards INNER JO...	371 row(s) returned
✓	48 21:26:48	SELECT DATE(timestamp) AS fechas_ventas, SUM(amount) AS total_ventas FROM transaction GROUP BY 1 O...	5 row(s) returned
✓	49 21:29:36	SELECT DATE(timestamp) AS fechas_ventas, SUM(amount) AS total_ventas FROM transaction GROUP BY DA...	5 row(s) returned

En esta consulta, selecciono el timestamp con la función DATE() que recorta la hora para mostrar solo la fecha, y la suma de ampunt con el alias total_ventas de la tabla transaction. Agrupo por la fecha y ordeno de manera descendente por el total ventas, y limito a 5 para sacar los 5 días con mejores ingresos.

Ejercicio 2

```

48 -- Presenta el nom, telèfon, país, data i amount, d'aquelles empreses que van realitzar transaccions
49 -- amb un valor comprès entre 350 i 400 euros i en alguna d'aquestes dates:
50 -- 29 d'abril del 2015, 20 de juliol del 2018 i 13 de març del 2024.
51 -- Ordena els resultats de major a menor quantitat.
52 • SELECT c.company_name, c.phone, c.country, DATE(t.timestamp) AS fecha_transaccion, t.amount
53 FROM companies c
54 INNER JOIN transaction t
55 ON c.company_id=t.business_id
56 WHERE t.amount BETWEEN 350 AND 400
57 AND DATE(t.timestamp) IN ('2015-04-29','2018-07-20','2024-03-13')
58 ORDER BY t.amount DESC;

```

company_name	phone	country	fecha_transaccion	amount
Fringilla LLC	+1 150 784 1716	New Zealand	2015-04-29	390.33
Auctor Mauris Vel LLP	+32 133 447 8239	United States	2024-03-13	383.58
Mauris Institute	+75 473 619 9472	Sweden	2015-04-29	369.20
Pede Cum Ltd	+26 625 798 6126	Norway	2018-07-20	358.64
Netus Et Malesuada Ltd	+52 818 402 1638	Netherlands	2024-03-13	352.21
Tempor Diam Institute	+89 848 141 3136	Netherlands	2024-03-13	350.71

Result 32 × Read Only

Output

Action Output

#	Time	Action	Message
✓ 47	21:21:59	SELECT credit_cards.iban, ROUND(AVG(transaction.amount),2) AS media_gasto FROM credit_cards INNER JO...	371 row(s) returned
✓ 48	21:26:48	SELECT DATE(timestamp) AS fechas_ventas, SUM(amount) AS total_ventas FROM transaction GROUP BY 1 O...	5 row(s) returned
✓ 49	21:29:36	SELECT DATE(timestamp) AS fechas_ventas, SUM(amount) AS total_ventas FROM transaction GROUP BY DA...	5 row(s) returned
✓ 50	21:33:35	SELECT c.company_name, c.phone, c.country, DATE(t.timestamp) AS fecha_transaccion, t.amount FROM com...	6 row(s) returned

En esta consulta, selecciono de la tabla companies el company_name, teléfono, país, la fecha con la función DATE() para recortar la hora y quedarnos solo con la fecha, y el amount de la tabla transaction, de la tabla companies y con un inner join junto la tabla transaction y las uno con un ON mediante la llave primaria y foránea. Añado una restricción WHERE el amount se encuentra entre 350 y 400, y la fecha se encuentre entre las indicadas con el IN. Finalmente, ordeno por amount descendente. Me salen 6 filas.

Ejercicio 3

```

62 -- però el departament de recursos humans és exigent i vol un llistat de les empreses on especifiquis si tenen
63 -- igual o més de 400 transaccions o menys.
64 • SELECT c.company_name,COUNT(t.id) AS total_transacciones,
65     CASE
66         WHEN COUNT(t.id) >= 400 THEN '400 o més transaccions'
67         ELSE 'Menys de 400 transaccions'
68     END AS capacidad_operativa
69 FROM companies c
70 INNER JOIN transaction t
71 ON c.company_id=t.business_id
72 GROUP BY c.company_name
73 ORDER BY total_transacciones;

```

Result Grid			
company_name	total_transacciones	capacidad_operativa	
► Lorem Eu Incorporated	380	Menys de 400 transaccions	
Fringilla LLC	397	Menys de 400 transaccions	
Nec Luctus LLC	399	Menys de 400 transaccions	
Dui Quis Institute	402	400 o més transaccions	
Risus Associates	407	400 o més transaccions	
Magna A Neque Industries	410	400 o més transaccions	
Amet Luctus Vulputate Foundation	411	400 o més transaccions	

Result 33 x

Output			
#	Time	Action	Message
✓ 46	21:20:44	SELECT * FROM users u WHERE (SELECT COUNT(id) AS total_transacciones FROM transaction t WHERE t....	4 row(s) returned
✓ 47	21:21:59	SELECT credit_cards.iban, ROUND(AVG(transaction.amount),2) AS media_gasto FROM credit_cards INNER JO...	371 row(s) returned
✓ 48	21:26:48	SELECT DATE(timestamp) AS fechas_ventas, SUM(amount) AS total_ventas FROM transaction GROUP BY 1 O...	5 row(s) returned
✓ 49	21:29:36	SELECT DATE(timestamp) AS fechas_ventas, SUM(amount) AS total_ventas FROM transaction GROUP BY DA...	5 row(s) returned
✓ 50	21:33:35	SELECT c.company_name, c.phone, c.country, DATE(t.timestamp) AS fecha_transaccion, t.amount FROM com...	6 row(s) returned
✓ 51	21:37:58	SELECT c.company_name,COUNT(t.id) AS total_transacciones, CASE WHEN COUNT(t.id) >= 400 THEN '400 ...	100 row(s) returned

En esta consulta, selecciono el company_name de companies, cuento el id de transaction y hago un case con las condiciones when y else. En el WHEN, cuento el id de transaction y le indico que si es mayor o igual a 400 le ponga 400 o más transacciones. Si es diferente a la condición que ponga menos de 400 transacciones y lo nombro capacidad_operativa.

Selecciono las tablas companies y transaction unida con inner join y llave primaria y foránea con un ON, agrupo por el company_name y lo ordeno por el total_transacciones. Me devuelve 100 filas.

Ejercicio 4

```

75 -- Elimina de la taula transaction el registre amb ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de dades.
76 • SELECT * FROM transaction WHERE id='000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';

```

	id	card_id	business_id	timestamp	amount	declined	product_ids	user_id	lat	longitude
▶	000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	Ccs-5019	b-2370	2016-12-21 20:07:18	172.56	0	66, 69, 87	438	41.5972	12.2218
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

transaction 34 x

Output

Action Output

#	Time	Action	Message
✓ 47	21:21:59	SELECT credit_cards.iban, ROUND(AVG(transaction.amount),2) AS media_gasto FROM credit_cards INNER JO...	371 row(s) returned
✓ 48	21:26:48	SELECT DATE(timestamp) AS fechas_ventas, SUM(amount) AS total_ventas FROM transaction GROUP BY 1 O...	5 row(s) returned
✓ 49	21:29:36	SELECT DATE(timestamp) AS fechas_ventas, SUM(amount) AS total_ventas FROM transaction GROUP BY DA...	5 row(s) returned
✓ 50	21:33:35	SELECT c.company_name, c.phone, c.country, DATE(t.timestamp) AS fecha_transaccion, t.amount FROM com...	6 row(s) returned
✓ 51	21:37:58	SELECT c.company_name,COUNT(t.id) AS total_transacciones, CASE WHEN COUNT(t.id) >= 400 THEN '400 ...	100 row(s) returned
✓ 52	21:42:17	SELECT * FROM transaction WHERE id='000447FE-B650-4DCF-85DE-C7ED0EE1CAAD' LIMIT 0, 1000	1 row(s) returned

En primer lugar, reviso que el id exista, y me devuelve la fila con el id requerido.

```

75 -- Elimina de la taula transaction el registre amb ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de dades.
76 • SELECT * FROM transaction WHERE id='000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
77 • DELETE FROM transaction
78 WHERE id='000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
79 • SELECT * FROM transaction WHERE id='000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';

```

	id	card_id	business_id	timestamp	amount	declined	product_ids	user_id	lat	longitude	discount_amount	tax_amount	shipping_amount
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

transaction 35 x

Output

Action Output

#	Time	Action	Message
✓ 49	21:29:36	SELECT DATE(timestamp) AS fechas_ventas, SUM(amount) AS total_ventas FROM transaction GROUP BY DA...	5 row(s) returned
✓ 50	21:33:35	SELECT c.company_name, c.phone, c.country, DATE(t.timestamp) AS fecha_transaccion, t.amount FROM com...	6 row(s) returned
✓ 51	21:37:58	SELECT c.company_name,COUNT(t.id) AS total_transacciones, CASE WHEN COUNT(t.id) >= 400 THEN '400 ...	100 row(s) returned
✓ 52	21:42:17	SELECT * FROM transaction WHERE id='000447FE-B650-4DCF-85DE-C7ED0EE1CAAD' LIMIT 0, 1000	1 row(s) returned
✓ 53	21:43:57	DELETE FROM transaction WHERE id='000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) affected
✓ 54	21:43:57	SELECT * FROM transaction WHERE id='000447FE-B650-4DCF-85DE-C7ED0EE1CAAD' LIMIT 0, 1000	0 row(s) returned

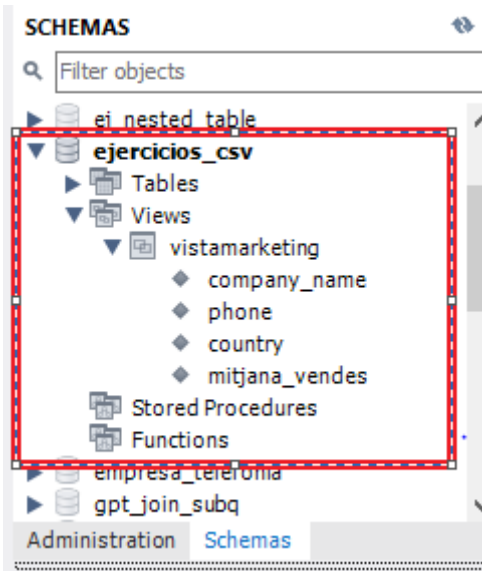
Elimino el id con DELETE FROM transaction WHERE id equivalga al solicitado en el enunciado, y lanzo de nuevo la consulta para ver si ha funcionado. Vemos que ha sido eliminada.

Ejercicio 5

-- La secció de màrqueting desitja tenir accés a informació específica per a realitzar anàlisi i estratègies
-- efectives. S'ha sol·licitat crear una vista que proporcioni detalls clau sobre les companyies
-- i les seves transaccions. Serà necessària que creis una vista anomenada VistaMarketing que contingui
-- la següent informació: Nom de la companyia. Telèfon de contacte. País de residència.
-- Mitjana de compra realitzat per cada companyia. Presenta la vista creada, ordenant les dades de major a menor
-- mitjana de compra.

```
CREATE VIEW VistaMarketing AS
SELECT c.company_name, c.phone, c.country, ROUND(AVG(t.amount),2) AS mitjana_vendes
FROM companies c
INNER JOIN transaction t
ON c.company_id=t.business_id
WHERE t.declined=0
GROUP BY c.company_name, c.phone, c.country
ORDER BY mitjana_vendes DESC;
```

Creo una vista llamada VistaMarketing, y selecciono el company_name, teléfono, país de companies, el average de amount de la tabla transaction, y uno las tablas companies y transaction, con una restricción donde no se haya rechazado ninguna transacción, agrupo por company_name, teléfono y país, y ordeno de manera descende por media de ventas.



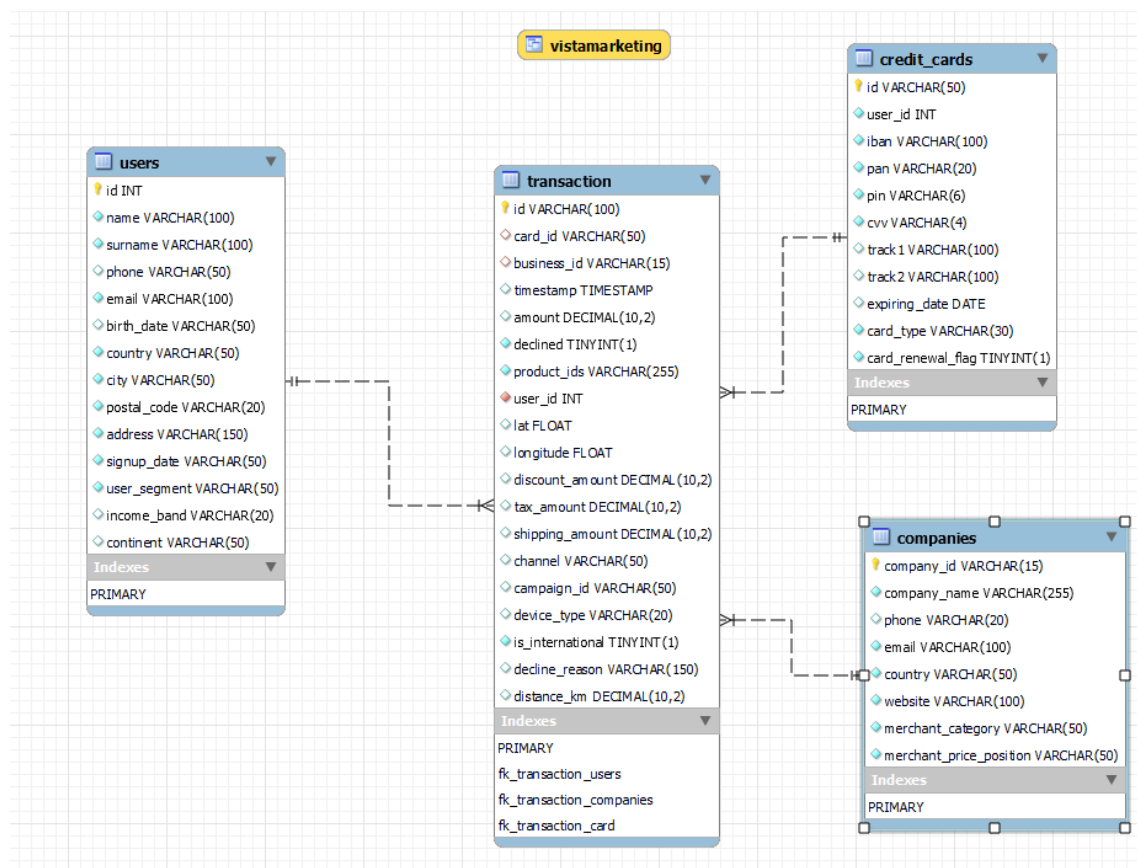
Veo que la vista se ha creado correctamente con los parámetros requeridos.

```
1 • SELECT * FROM ejercicios_csv.vistamarketing;
```

Result Grid					Filter Rows:	Export:	Wrap Cell Content:
	company_name	phone	country	mitjana_vendes			
▶	Ac Fermentum Incorporated	+64 601 244 6320	Germany	330.71			
	Ut Semper Foundation	+10 283 500 6449	Sweden	307.30			
	Dolor Vitae Limited	+6 421 567 9195	France	306.21			
	Orci Adipiscing Limited	+60 897 390 5820	United Kingdom	305.87			
	Nunc In Foundation	+13 267 741 2619	Italy	305.67			
	Maecenas Malesuada Fringilla Inc.	+51 673 540 5289	Netherlands	304.49			

Output				
Action Output				
#	Time	Action	Message	
✓ 53	21:43:57	DELETE FROM transaction WHERE id='000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) affected	
✓ 54	21:43:57	SELECT * FROM transaction WHERE id='000447FE-B650-4DCF-85DE-C7ED0EE1CAAD' LIMIT 0, 1000	0 row(s) returned	
✓ 55	21:50:21	SELECT * FROM ejercicios_csv.vistamarketing LIMIT 0, 1000	100 row(s) returned	

Devuelve 100 filas.



Veo que se ha creado la vista correctamente.

Nivel 3

Ejercicio 1

```
3 WITH estat_targetes AS(
4   SELECT id, card_id, declined,
5         ROW_NUMBER() OVER(PARTITION BY card_id ORDER BY id DESC) AS orden_transaccion
6   FROM transaction
7 ),
8 resum_estat AS(
9   SELECT card_id,
10         CASE
11           WHEN SUM(CASE WHEN declined = 0 THEN 1 ELSE 0 END) > 0 THEN 'Activa'
12           ELSE 'Inactiva'
13         END AS tarjetas_activas
14   FROM estat_targetes
15  WHERE orden_transaccion <=3
16  GROUP BY card_id
17 )
18 SELECT COUNT(*) AS total_tarjetas_activas
19 FROM resum_estat
20 WHERE tarjetas_activas = 'Activa';
```

Result Grid

total_tarjetas_activas
5000

Result 5 x

Output

Action Output

#	Time	Action	Message
8	13:27:18	WITH estat_targetes AS(SELECT t.id, t.card_id, t.declined, ROW_NUMBER() OVER(PARTITION BY card_id O...	1 row(s) returned
9	13:28:34	WITH estat_targetes AS(SELECT id, card_id, declined, ROW_NUMBER() OVER(PARTITION BY card_id ORD...	1 row(s) returned

Creo una CTE (Common Table Expression) y selecciono el id, card_id y declined de la tabla transacción. Aplico la función ROW_NUMBER() y creo una función ventana con OVER() y usando PARTITION BY card_id, lo que hago es dividir el card_id en grupos independientes (que más adelante filtraré), ordeno por id descendente y le llamo orden_transaccion. Creo una subconsulta que llamo resum_estat, selecciono el card_id y aquí es cuando hago el filtro de las tarjetas con un case end. En el WHEN, hago una suma de las 3 últimas transacciones, y un contador que dice si las transacciones son mayores a 0 (aprobadas) entonces la tarjeta está activa, en caso contrario, inactiva. Lo llamo tarjetas_activas y selecciono el nombre de la tabla (CTE creada) como estat_targetes y le indico que busque las 3 últimas transacciones y lo agrupo por card_id. Finalmente, selecciono con un COUNT() de todo y lo guardo como total_tarjetas_activas, de la subconsulta resum_estat y filtro por tarjetas_activas='Activa'.

Ejercicio 2

```
32 CREATE TABLE IF NOT EXISTS products(  
33   id INT PRIMARY KEY,  
34   product_name VARCHAR(150),  
35   price VARCHAR(20),  
36   colour VARCHAR(30),  
37   weight DECIMAL(5,2),  
38   warehouse_id VARCHAR(10),  
39   category VARCHAR(50),  
40   brand VARCHAR(50),  
41   cost VARCHAR(20),  
42   launch_date DATE  
43 );  
44  
45 LOAD DATA  
46 INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__products.csv'  
47 INTO TABLE products  
48 FIELDS TERMINATED BY ','  
49 LINES TERMINATED BY '\n'  
50 IGNORE 1 LINES;
```

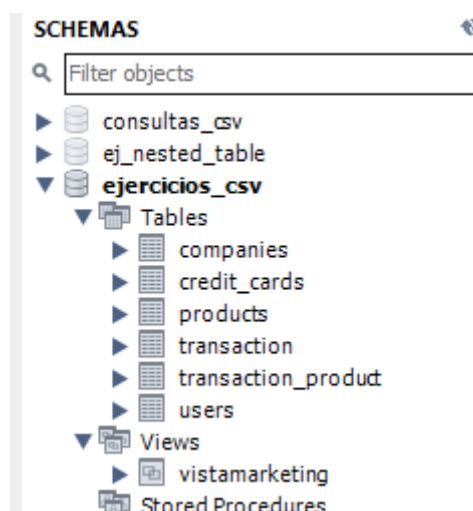
Output

Action Output

#	Time	Action	Message
✓ 13	14:12:53	SELECT * FROM ejercicios_csv.products LIMIT 0, 1000	100 row(s) returned
✓ 14	14:13:09	SELECT COUNT(*) FROM products LIMIT 0, 1000	1 row(s) returned

Creo la tabla products con los atributos del documento .csv. Un id que hace de llave primaria, y otros datos que definen los productos (como el nombre, precio, color, peso, almacén, categoría, marca y fecha de lanzamiento).

Añado los datos con LOAD DATA INFILE de la carpeta ProgramData de SQL en la tabla products, indico que los campos terminan con ',' y que hay un salto de línea. Elimino las cabeceras de los documentos.



Aparece la tabla creada en la base de datos.

2 • `SELECT COUNT(*) FROM products;`

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [F1](#)

COUNT(*)
100

Result 2 x

Output

Action Output

#	Time	Action	Message
✓ 13	14:12:53	SELECT * FROM ejercicios_csv.products LIMIT 0, 1000	100 row(s) returned
✓ 14	14:13:09	SELECT COUNT(*) FROM products LIMIT 0, 1000	1 row(s) returned

Cuento toda la tabla, y me da un resultado de 100 productos.

21 • `SELECT p.id, p.product_name, COUNT(*) AS veces_vendido`

22 `FROM transaction t`

23 `INNER JOIN products p`

24 `ON FIND_IN_SET(p.id, REPLACE(t.product_ids, ',', '')) > 0`

25 `GROUP BY p.id, p.product_name`

26 `ORDER BY p.id;`

27

28

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [F1](#)

id	product_name	veces_vendido
1	Direwolf Stannis	2468
2	Tarly Stark	2573
3	duel tourney Lannister	2528
4	warden south duel	2584
5	skywalker ewok	2548

Result 9 x

Output

Action Output

#	Time	Action	Message
✓ 18	14:34:56	SELECT p.id, p.product_name, COUNT(*) AS veces_vendido FROM transaction t INNER JOIN products p ON ...	100 row(s) returned
✓ 19	14:35:26	SELECT p.id, p.product_name, COUNT(*) AS veces_vendido FROM transaction t INNER JOIN products p ON ...	100 row(s) returned

Selecciono el id y el nombre del producto de la tabla products y cuento todo como veces_vendido. Uno las tablas transaction con products, pero en lugar de hacer un ON y unir llave primaria con foránea, uso la función FIND_IN_SET(). En el archivo original, la columna id_products, los id's se encuentran juntos en la misma celda, separados por coma

y espacios. Lo que hace REPLACE es precisamente limpiar esos datos, eliminar los espacios y los cambia por nada, para que pueda leerlos. La función FIND_IN_SET busca directamente dentro de esa lista limpia, compruebo que el resultado es mayor de 0, lo que implica que el producto aparece en la lista (en caso contrario, el producto no existe). Por último, agrupo por id y nombre producto de la tabla producto y ordeno por el id de la tabla producto. Retorna el número de ventas de cada producto con su id.